

서블릿, JSP, MVC2, Spring MVC 정리

Spring Day1 읽기용 정리본 - 발표 슬라이드가 아니라 개념 복습용 자료

0. 전체 흐름 먼저 보기

Java 웹 개발은 처음에는 서블릿이 요청 처리와 HTML 응답을 모두 담당했지만, 코드가 길어지고 화면 수정이 어려워지면서 JSP와 역할을 나누기 시작했다. 그 다음에는 요청을 한 곳에서 받아 분배하는 Front Controller 방식이 등장했고, 이 구조를 표준화한 것이 MVC2 패턴이다. Spring MVC는 MVC2 구조를 프레임워크로 제공해서 개발자가 반복 코드를 적게 쓰고 컨트롤러의 핵심 로직에 집중하게 해 준다.

큰 흐름

서블릿 단독 처리 -> 서블릿 + JSP -> forward/include -> Front Controller -> MVC2 -> Spring MVC

1. 서블릿
2. JSP
3. 서블릿 forward
4. 서블릿 include
5. Front Controller 패턴
6. MVC2 패턴
7. Spring MVC
8. `@RequestParam`
9. `@ModelAttribute`

1. 서블릿 Servlet

서블릿은 웹 서버에서 실행되는 Java 클래스다. 브라우저가 URL로 요청을 보내면, 톰캣 같은 WAS가 요청을 받아 서블릿의 메서드를 실행한다. 서블릿은 요청 데이터를 읽고, 필요한 처리를 한 뒤 응답을 만든다.

서블릿의 역할

- 클라이언트 요청을 받는다.
- 요청 파라미터를 읽는다. 예: `request.getParameter("id")`
- 비즈니스 로직 또는 서비스 객체를 호출한다.
- 응답을 직접 만들거나 JSP로 넘긴다.

기본 예시

```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String id = request.getParameter("id");

    response.setContentType("text/html; charset=UTF-8");
    PrintWriter out = response.getWriter();
    out.println("<h1>" + id + "님 환영합니다</h1>");
}
```

서블릿에서 HTML을 직접 출력하면 Java 코드 안에 HTML 문자열이 계속 들어간다. 그래서 화면이 커질수록 읽기 어렵고 수정하기도 어렵다.

2. JSP

JSP는 HTML 안에 Java 코드를 사용할 수 있게 만든 서버 화면 파일이다. 실행될 때 내부적으로 서블릿으로 변환되어 처리된다. 그래서 JSP도 결국 서버에서 실행되고, 실행 결과로 만들어진 HTML이 브라우저에 전달된다.

JSP의 역할

- HTML 화면을 작성한다.
- 서블릿이나 컨트롤러가 request에 담아 준 데이터를 출력한다.
- JSTL, EL을 사용해서 반복 출력이나 조건 출력을 처리한다.

JSP 예시

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<h2>회원 정보</h2>
아이디: ${id}
```

JSP는 화면 담당으로 쓰는 것이 좋다. 계산, DB 처리, 복잡한 조건 판단은 서블릿이나 컨트롤러, 서비스 계층에서 처리하고 JSP는 출력에 집중하는 구조가 유지보수에 좋다.

3. 서블릿 forward

`forward` 는 현재 요청을 서버 내부에서 다른 자원으로 넘기는 기능이다. 보통 서블릿이 요청을 처리하고, 결과 데이터를 `request`에 담은 뒤 JSP로 넘길 때 사용한다. 브라우저 입장에서는 URL이 바뀌지 않는다.

forward 흐름

브라우저 요청 -> 서블릿 처리 -> request에 데이터 저장 -> JSP로 forward -> JSP가 화면 출력

forward 예시

```
request.setAttribute("message", "서블릿에서 만든 데이터");

RequestDispatcher dispatcher =
    request.getRequestDispatcher("/WEB-INF/views/result.jsp");

dispatcher.forward(request, response);
```

forward의 특징

- 같은 request 객체가 유지된다.
- `request.setAttribute()` 로 담은 값을 JSP에서 꺼낼 수 있다.
- 브라우저 주소창의 URL은 처음 요청한 URL 그대로다.
- 서블릿은 처리 담당, JSP는 화면 담당으로 역할을 나누기 좋다.

4. 서블릿 include

`include` 는 다른 자원의 실행 결과를 현재 응답 안에 포함하는 기능이다. 현재 흐름을 완전히 넘기는 `forward`와 달리, `include`는 다른 자원의 결과를 끼워 넣고 다시 현재 서블릿으로 돌아온다.

include 흐름

현재 서블릿 실행 -> 다른 JSP/서블릿 결과 포함 -> 다시 현재 서블릿 실행 계속 -> 응답 완료

include 예시

```
RequestDispatcher dispatcher =
    request.getRequestDispatcher("/menu.jsp");

dispatcher.include(request, response);
```

구분	forward	include
목적	처리 결과를 다른 자원에 맡김	다른 자원의 결과를 현재 응답에 포함

구분	forward	include
흐름	넘긴 뒤 JSP가 최종 응답 담당	포함 후 원래 서블릿/JSP로 돌아옴
주 사용 예	서블릿에서 JSP로 화면 이동	공통 메뉴, 헤더, 푸터 포함

5. Front Controller 패턴

Front Controller는 모든 요청을 하나의 대표 컨트롤러가 먼저 받는 구조다. 대표 컨트롤러는 요청 URL을 보고 실제 일을 처리할 컨트롤러를 찾아 실행한다. 공통 처리와 요청 분배를 한 곳에서 관리할 수 있다는 장점이 있다.

왜 필요한가?

- 여러 서블릿마다 반복되는 공통 코드가 많아진다.
- 인코딩 처리, 로그인 확인, 예외 처리 같은 공통 작업을 한 곳에서 처리하고 싶다.
- URL과 실제 처리 객체의 연결을 체계적으로 관리하고 싶다.

Front Controller 흐름

브라우저 요청 -> Front Controller -> 알맞은 Controller 선택 -> Model 생성 -> View 선택 -> JSP 응답

간단한 구조 예시

```
String uri = request.getRequestURI();

Controller controller = null;

if (uri.equals("/member/list")) {
    controller = new MemberListController();
} else if (uri.equals("/member/detail")) {
    controller = new MemberDetailController();
}

String viewName = controller.execute(request, response);
request.getRequestDispatcher(viewName).forward(request, response);
```

Spring MVC의 DispatcherServlet 이 바로 Front Controller 역할을 한다.

6. MVC2 패턴

MVC2는 웹 애플리케이션에서 Model, View, Controller의 역할을 분리하는 구조다. MVC1은 JSP가 요청 처리와 화면 출력을 많이 담당하지만, MVC2는 컨트롤러가 요청을 처리하고 JSP는 화면 출력에 집중한다.

역할	담당	예시
Model	데이터와 처리 결과	회원 목록, 책 정보, 계산 결과, 서비스/DAO 결과
View	화면 출력	JSP, HTML, JSTL, EL
Controller	요청 접수, 파라미터 수집, 모델 생성, 뷰 선택	Servlet, Spring Controller

MVC2 흐름

요청 -> Controller -> Model 준비 -> View 이름 결정 -> forward -> JSP 출력

MVC2의 장점

- 화면과 처리 로직이 분리되어 유지보수가 쉽다.
- JSP에 Java 코드가 줄어든다.
- 컨트롤러, 서비스, DAO처럼 역할별로 코드를 나눌 수 있다.
- URL 요청 처리 구조가 명확해진다.

7. Spring MVC

Spring MVC는 MVC2와 Front Controller 구조를 Spring이 제공하는 웹 프레임워크다. 개발자는 직접 Front Controller를 만들지 않고, Spring이 제공하는 `DispatcherServlet` 을 사용한다. 실제 요청 처리는 개발자가 만든 Controller 메서드가 담당한다.

Spring MVC 핵심 흐름

브라우저 요청 -> DispatcherServlet -> HandlerMapping -> Controller 메서드 실행 -> Model 준비 -> ViewResolver -> JSP 렌더링 -> 응답

Spring MVC에서 달라지는 점

- 서블릿을 직접 여러 개 만들지 않아도 된다.
- `@Controller` 클래스를 만들고 메서드 단위로 URL을 매핑할 수 있다.
- `@GetMapping`, `@PostMapping` 으로 요청 방식을 구분할 수 있다.
- 파라미터 수집을 `@RequestParam`, `@ModelAttribute` 로 편하게 처리한다.
- 컨트롤러는 문자열로 뷰 이름을 반환할 수 있다.

Controller 예시

```
@Controller
public class HelloController {

    @GetMapping("/hi")
    public String hi(Model model) {
        model.addAttribute("data", "Spring MVC");
        return "view";
    }
}
```

위 코드에서 `return "view"` 는 보통 `/WEB-INF/views/view.jsp` 같은 JSP로 연결된다. 이 연결은 `ViewResolver` 설정이 담당한다.

8. @RequestParam

`@RequestParam` 은 요청 파라미터 하나하나를 컨트롤러 메서드의 매개변수로 받을 때 사용한다. 예를 들어 `/login?id=test&pw=1234` 요청이 들어오면 `id`와 `pw` 값을 바로 받을 수 있다.

기존 서블릿 방식

```
String id = request.getParameter("id");
String pw = request.getParameter("pw");
```

Spring MVC 방식

```
@GetMapping("/login")
public String login(
    @RequestParam("id") String id,
    @RequestParam("pw") String pw,
    Model model) {

    model.addAttribute("id", id);
    model.addAttribute("pw", pw);
    return "loginResult";
}
```

자주 쓰는 옵션

```
@RequestParam(value = "num", defaultValue = "1") int num
@RequestParam(value = "keyword", required = false) String keyword
```

옵션	의미
<code>value</code>	요청 파라미터 이름

옵션	의미
<code>required</code>	필수 여부. 기본값은 true
<code>defaultValue</code>	값이 없을 때 사용할 기본값

9. @ModelAttribute

`@ModelAttribute` 는 여러 요청 파라미터를 하나의 객체에 자동으로 담을 때 사용한다. 파라미터 이름과 객체의 필드 이름이 같으면 Spring이 setter 또는 생성 규칙을 이용해 값을 넣어 준다.

요청 예시

```
/user?id=test&pw=1234&name=홍길동
```

DTO 또는 VO 클래스

```
public class User {
    private String id;
    private String pw;
    private String name;

    public String getId() { return id; }
    public void setId(String id) { this.id = id; }

    public String getPw() { return pw; }
    public void setPw(String pw) { this.pw = pw; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Controller 예시

```
@GetMapping("/user")
public String user(@ModelAttribute User user, Model model) {
    model.addAttribute("user", user);
    return "userView";
}
```

단순히 `User user` 처럼 적어도 동작하는 경우가 있지만, 수업 초기에는 `@ModelAttribute` 를 붙여서 의도를 분명히 보는 것이 이해하기 좋다.

@RequestParam과 @ModelAttribute 비교

구분	@RequestParam	@ModelAttribute
받는 대상	파라미터 하나 또는 몇 개	여러 파라미터를 객체 하나로 묶음
사용 예	<code>id</code> , <code>pw</code> , <code>num</code>	<code>User</code> , <code>Book</code> , <code>Score</code>
좋은 상황	값이 적고 단순할 때	폼 입력값이 많고 객체로 다루고 싶을 때

10. 전체 비교 정리

개념	핵심 의미	기억할 점
서블릿	Java로 요청을 처리하는 서버 프로그램	요청 처리 담당
JSP	HTML 중심의 서버 화면 파일	화면 출력 담당
forward	서버 내부에서 요청을 다른 자원으로 넘김	서블릿에서 JSP로 이동할 때 자주 사용
include	다른 자원의 결과를 현재 응답에 포함	공통 화면 조각 포함에 사용
Front Controller	모든 요청을 한 대표 컨트롤러가 먼저 받음	공통 처리와 요청 분배를 한 곳에서 관리
MVC2	Model, View, Controller 역할 분리	유지보수와 확장성이 좋아짐
Spring MVC	Spring이 제공하는 MVC2 웹 프레임워크	DispatcherServlet이 Front Controller 역할
@RequestParam	요청 파라미터를 변수로 받음	값이 적을 때 편함
@ModelAttribute	요청 파라미터를 객체로 묶음	폼 데이터가 많을 때 편함

11. 수업 실습과 연결해서 보기

수업에서 만드는 `/calc`, `/dan`, `/star`, `/divisor` 같은 요청은 모두 같은 구조로 생각할 수 있다.

URL 요청 -> Controller 메서드 실행 -> 파라미터 수집 -> 계산 또는 객체 생성 -> Model에 결과 저장 -> JSP에서 출력

예: 원하는 단 출력

```
@GetMapping("/dan")
public String dan(@RequestParam("num") int num, Model model) {
    model.addAttribute("num", num);
    return "danView";
}
```

예: 책 정보 받기

```
@GetMapping("/book")
public String book(@ModelAttribute Book book, Model model) {
    model.addAttribute("book", book);
}
```

```
return "bookView";  
}
```

12. 꼭 기억할 문장

- 서블릿은 요청 처리 담당, JSP는 화면 출력 담당이다.
- forward는 서블릿이 만든 데이터를 JSP로 넘겨 화면을 보여줄 때 많이 쓴다.
- Front Controller는 모든 요청을 먼저 받는 대표 컨트롤러다.
- MVC2는 Controller, Model, View의 역할을 나누는 구조다.
- Spring MVC의 DispatcherServlet은 Front Controller 역할을 한다.
- `@RequestParam` 은 날개 값, `@ModelAttribute` 는 객체로 받는 데 적합하다.

작성 위치: C:\AcornFile\AcornFileMain\Spring\수업자료\SpringDay1